Return of Coppersmith's Attack (ROCA)

Prepared by:
230001013 – Sai Ram Swaroop Teja
230001019 – Tanmai Sai
230001023 – Chennakeshava
230004005 – Ansh Jain
Department of Computer Science and Engineering
IIT Indore

November 15, 2025

Contents

1 Introduction **3**
1.1 Overview . . . 3
1.2 RSA Algorithm . . . 3

2 Generation of Primes **3**
2.1 Generation of Primes in RSALib . . . 3
2.2 Explaining the weakness of this generation . . . 4

3 Detection of Vulnerable Keys **6**
3.1 Special Property of the Modulus N . . . 6
3.2 Restricted Remainders . . . 6
3.3 Explaning the behaviour of N . . . 7
3.4 Fingerprinting Vulnerable Keys . . . 7
3.5 Discrete Log Based Fingerprinting . . . 8

4 Testing whether a key is vulnerable to ROCA attack **9**

5 The Attack Principle **11**
5.1 Mapping This Condition to the ROCA Attack Scenario . . . 11
5.2 Checking whether can we apply CHG . . . 12
5.3 Transforming the Problem to CHG . . . 14
5.4 CHG Parameters . . . 15
5.5 The Polynomial $f(x)$ and the Modulus N . . . 15
5.6 Lower Bound β . . . 15
5.7 Parameters m and t . . . 15
5.8 The Upper Bound X . . . 16
5.9 Lattice Construction in CHG . . . 18

6 Optimizations **19**
6.1 Reducing the Search Space for a . . . 19
6.2 β Parameter Issue After Reduction of the Search Space for a . . . 20
6.3 Optimizing the Value M . . . 20

6.4 Optimizing the Values M' and m 21

7 Implementation of the Return of Coppersmith's Attack or ROCA using CHG Algorithm 22

8 Countermeasures and Mitigation 25
8.1 Eliminating the Vulnerable Prime Structure 25
8.2 Firmware and Library Updates 26
8.3 Key Revocation and Regeneration 26
8.4 Vendor and Ecosystem Response 26
8.5 Long-Term Lessons for Cryptographic Engineering 26
8.6 Summary 26

9 Conclusion 27

1 Introduction

1.1 Overview

- The ROCA weakness takes advantage of a flaw in how certain RSA public keys were constructed, allowing an attacker to recover the private key by factoring the public modulus.
- ROCA stands for "Return Of Coppersmith's Attack". It was discovered in February 2017 by a team of Czech researchers.
- The root cause is a weak RSA key-generation routine in a library (commonly referred to as RSALib) from Infineon Technologies AG. That library was embedded in many smartcards, so the flaw had wide-reaching consequences

1.2 RSA Algorithm

- Key Generation:
  Select two large prime numbers p_1 and p_2, compute $N = p_1 \times p_2$ and $\phi(N) = (p_1 - 1)(p_2 - 1)$.
  Choose a public exponent $1 < e < \phi(N)$ and find the private exponent d such that $d \times e \equiv 1 \pmod{\phi(N)}$.
  Public key is (N, e) and private key is (N, d).
- Encryption: Represent the message m as an integer $0 \leq m < N$ and compute ciphertext $c = m^e \bmod N$.
- Decryption: Receive the ciphertext c and compute $m = c^d \bmod N$.

2 Generation of Primes

2.1 Generation of Primes in RSALib

- In RSALib, the primes were chosen with a very particular structure (likely to speed up generation). Each prime p was generated in the form:

$$p = k \cdot M + (65537^{\,a} \bmod M)$$

- Here the integers k and a are randomly chosen during generation, and M is a **primorial** (the product of the first n primes):

$$M = P_n\# = \prod_{i=1}^{n} P_i = 2 \times 3 \times 5 \times 7 \times \cdots \times P_n$$

- The primorial M used depends on the desired RSA key length: larger RSA keys use a larger primorial. For example, RSALib uses $n = 39$ (so $M = P_{39}\#$) for key sizes in the 512–960 bit range.

Primorials used for different RSA key sizes

- The table below shows the various primorials used for each key size. Certain key sizes are not supported by the RSALib library.

Key Size	n	M
512−960	39	$P_{39}\#$
992−1952	71	$P_{71}\#$
1984−3936	126	$P_{126}\#$
3968−4096	225	$P_{225}\#$

2.2 Explaining the weakness of this generation

- The problem comes from the fact that M is large relative to the target prime size. For a 512-bit RSA key, each prime p is roughly 256 bits, The number of bits in the primorial is **computed below:-**

```python


from math import log
from Crypto.Util.number import getRandomNBitInteger, isPrime as
    is_prime, getPrime
from sympy import nextprime as next_prime


# ----------------------------------------------------------
# Compute primorial P_n#
# ----------------------------------------------------------
def compute_primorial(count):
    product = 1
    p = 1
    for _ in range(count):
        p = next_prime(p)
        product *= p
    return int(product)


# ----------------------------------------------------------
# Compute number of bits in primorial for given RSA key size
# ----------------------------------------------------------
def numbits(key_bits):
    if 512 <= key_bits <= 960:
        n = 39
    elif 992 <= key_bits <= 1952:
        n = 71
    elif 1984 <= key_bits <= 3936:
        n = 126
    elif 3968 <= key_bits <= 4096:
        n = 225
    else:
        return None

    M = compute_primorial(n)
    return round(log(M, 2))


# ----------------------------------------------------------
# Example usage
# ----------------------------------------------------------
if __name__ == "__main__":
    print("Bits of primorial for 512-bit key:", numbits(512))
```

```
PS C:\Users\desha> python -u "c:\Users\desha\Downloads\slide 7 - file0.py"
Bits of primorial for 512-bit key: 219
```

- So, the primorial M for $n = 39$ has about 219 bits. That leaves the multiplier k with only around $256 - 219 = 37$ bits of entropy.

- To bound a, consider the multiplicative subgroup generated by $g = 65537$ modulo M:

$$\{1,\ g,\ g^2,\ g^3,\ \ldots,\ g^{t-1}\} \pmod{M},$$

where t is the multiplicative order of g modulo M (the smallest positive integer such that $g^t \equiv 1 \pmod{M}$).

- **The value of t and the entropy of a are computed below**

```python
from sympy import primerange, factorint
from math import log2

def primorial(n):
    primes = list(primerange(1, 10_000))[:n]
    M = 1
    for p in primes:
        M *= p
    return M

def multiplicative_order(g, M, n):
    primes = list(primerange(1, 10_000))[:n]
    phi = 1
    for p in primes:
        phi *= (p - 1)

    factors = factorint(phi)

    t = phi
    for q, e in factors.items():
        for _ in range(e):
            if pow(g, t // q, M) == 1:
                t //= q
            else:
                break
    return t

g = 65537
n = 39

M = primorial(n)
print("Primorial M bits:", M.bit_length())

t = multiplicative_order(g, M, n)
print("Multiplicative Order t =", t)

entropy_a = log2(t)
print("Entropy of a (bits)  ", entropy_a)
```

```
(base) tanmaisai@hpenvy:~$ python3 file1.py
Primorial M bits: 220
Multiplicative Order t = 2454106387091158800
Entropy of a (bits) ≈ 61.08990350008742
```

- So, for the 512-bit RSA key, a has about 62 bits of entropy.
- Hence, each prime effectively has at most about $37 + 62 = 99$ bits of entropy, rather than the expected 256 bits.

- That drastically reduces the search space from 2^{256} possibilities to roughly 2^{99}, which is still large but amenable to far more sophisticated attacks than a blind brute force.

3 Detection of Vulnerable Keys

3.1 Special Property of the Modulus N

- Since both primes p_1 and p_2 follow the same form, we have:

$$N = p_1 \times p_2$$

$$= (k_1 M + (65537^{a_1} \bmod M)) \times (k_2 M + (65537^{a_2} \bmod M))$$

$$= k_1 k_2 M^2 + k_1 M(65537^{a_2} \bmod M) + k_2 M(65537^{a_1} \bmod M) + (65537^{a_1} \cdot 65537^{a_2} \bmod M)$$

$$= k_3 M + (65537^{a_3} \bmod M)$$

- where

$$k_3 = k_1 k_2 M \ + \ (k_1(65537^{a_2} \bmod M) + k_2(65537^{a_1} \bmod M))$$

and

$$a_3 = a_1 + a_2.$$

- This shows that the modulus N itself retains the same structural form as its prime factors.

3.2 Restricted Remainders

- When researchers examined numerous public keys generated by Infineon's RSALib, they noticed that N took only a small subset of possible remainders modulo small primes.
- For example:

$$N \bmod 11 \in \{1, 10\}$$

$$N \bmod 13 \in \{1, 3, 4, 9, 10, 12\}$$

$$N \bmod 17 \in \{1, 2, 4, 8, 9, 13, 15, 16\}$$

$$N \bmod 19 \in \{1, 4, 5, 6, 7, 9, 11, 16, 17\}$$

$$N \bmod 37 \in \{1, 10, 26\}$$

- If p_1 and p_2 were random primes, then the remainders of $N = p_1 \times p_2$ modulo any small prime p would be uniformly distributed among all $p - 1$ possible values.
- However, because of the structured generation process, N exhibits clear patterns that can be detected.

3.3 Explaning the behaviour of N

- We have just shown that

$$N = k_3 M + (65537^{a_3} \bmod M).$$

  If we take the modulus N modulo M, we get:

$$N \equiv 65537^{a_3} \pmod{M}.$$

- Now M is a multiple of each small prime p since it is a primorial. Hence it follows that:

$$N \equiv 65537^{a_3} \pmod{p}.$$

- The set of all possible remainders is obtained by letting a_3 take all its allowed values.

$$\text{Possible remainders of } N \bmod p = \{65537^0,\ 65537^1,\ 65537^2,\ \ldots,\ 65537^{(\text{order}-1)}\} \bmod p$$

- That is, the group of remainders that N can take modulo p corresponds exactly to the multiplicative subgroup generated by 65537 mod p.
- We can compute the multiplicative order of 65537 mod p and hence find all the possible remainders of N mod p.
- Computing it for $p = 11$, 13, 17, 19, 37 and verify whether the remainders are restricted or not!

```python
from sympy import factorint

def multiplicative_order_mod_p(g, p):
    phi = p - 1
    factors = factorint(phi)
    order = phi
    for q, e in factors.items():
        for _ in range(e):
            if pow(g, order // q, p) == 1:
                order //= q
            else:
                break
    return order

g = 65537
primes = [11, 13, 17, 19, 37]

for p in primes:
    order = multiplicative_order_mod_p(g, p)
    residues = {pow(g, a, p) for a in range(order)}
    print(f"p = {p}, order = {order}, residues = {sorted(residues)}
        ")
```

3.4 Fingerprinting Vulnerable Keys

- Because N exhibits these predictable remainders, a simple statistical test can flag potentially vulnerable keys.
- The test checks whether N mod p falls into a known restricted set for several small primes.

```
(base) tanmaisai@hpenvy:~$ python3 file2.py
p = 11, order = 2, residues = [1, 10]
p = 13, order = 6, residues = [1, 3, 4, 9, 10, 12]
p = 17, order = 8, residues = [1, 2, 4, 8, 9, 13, 15, 16]
p = 19, order = 9, residues = [1, 4, 5, 6, 7, 9, 11, 16, 17]
p = 37, order = 3, residues = [1, 10, 26]
```

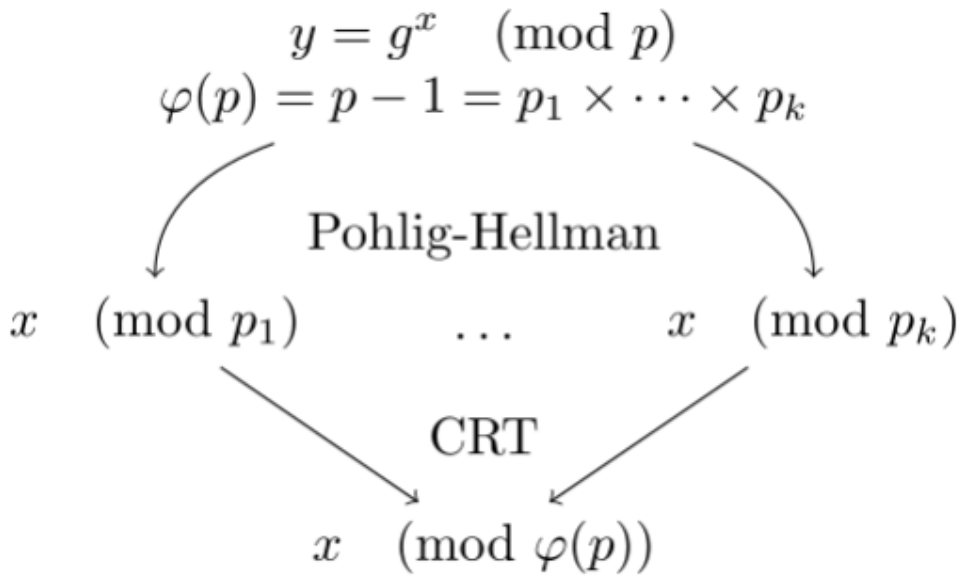
3.5 Discrete Log Based Fingerprinting

- A stronger, near–foolproof fingerprint method relies on computing the discrete logarithm. Since
$$N \equiv 65537^{a_3} \pmod{M},$$
we have
$$a_3 \equiv \log_{65537}(N) \pmod{M}.$$

- Normally, solving discrete logarithms is hard. However, because M is smooth (it is the product of small primes), the Pohlig–Hellman algorithm can efficiently solve it.

- Since M is composed entirely of small factors, the value $\phi(M)$ — the order of the multiplicative group $\mathbb{Z}_M^*$ — is also smooth.
Since
$$M = \prod_{i=1}^{n} P_i,$$
we have
$$\phi(M) = \prod_{i=1}^{n} (P_i - 1),$$
which also contains only small primes in its factorization, making it ideal for fast discrete log computation.

- The steps of the Pohlig–Hellman algorithm given below.



- For a 512-bit key: the size of group generated by 65537 as generator, $|G| = 2^{62}$ (we've computed the entropy of a earlier which comes out to be 62).

- The size of the entire group i.e. $\mathbb{Z}_M^*$ $|\mathbb{Z}_M^*| = |\phi(M)|$ is computed below.

```python
from sympy import primerange
from math import log2

n = 39

primes = list(primerange(1, 10000))[:n]

M = 1
for p in primes:
    M *= p

phi_M = 1
for p in primes:
    phi_M *= (p - 1)

print("phi(M) =", phi_M)
print("bit length =", phi_M.bit_length())
```

```
(base) tanmaisai@hpenvy:~$ python3 file3.py
phi(M) = 10386909671343413114146268913039653104541480138601136128000000000
bit length = 216
```

- So, $|\mathbb{Z}_M^*| = 2^{216}$.
- Hence, the probability of a random modulus N falling into G (the subgroup generated by 65537) is:

$$\frac{2^{62}}{2^{216}} = 2^{-154},$$

which is astronomically small, so the test almost never wrongly flags a secure key.

4 Testing whether a key is vulnerable to ROCA attack

```python
    # sage -python detect_roca.py

# Necessary imports / headers
from math import ceil, log, floor
from Crypto.Util.number import getRandomNBitInteger, isPrime as
    is_prime, getPrime
from sympy import nextprime as next_prime
from sage.all import Zmod   # requires running under sage -python

# ----- Primorial / weak-key helpers-----

def compute_primorial(count):
    product = 1
    prime_cursor = 1
    for _ in range(count):
        prime_cursor = next_prime(prime_cursor)
        product *= prime_cursor
    return int(product)

def derive_parameters(key_bits):
    if 512 <= key_bits <= 960:
        n = 39
    elif 992 <= key_bits <= 1952:
```

```
        n = 71
    elif 1984 <= key_bits <= 3936:
        n = 126
    elif 3968 <= key_bits <= 4096:
        n = 225
    else:
        return None
    M = compute_primorial(n)
    k_bits = key_bits // 2 - round(log(M, 2))
    g_order = Zmod(M)(65537).multiplicative_order()
    a_bits = ceil(log(g_order, 2))
    return M, k_bits, a_bits

def generate_weak_prime(M, k_bits, a_bits):
    p = 0
    while not is_prime(p):
        k_val = getRandomNBitInteger(k_bits)
        a_val = getRandomNBitInteger(a_bits)
        p = k_val * M + pow(65537, a_val, M)
    return p

def generate_weak_modulus(size_bits):
    params = derive_parameters(size_bits)
    if params is None:
        raise ValueError("Unsupported key size.")
    M, k_bits, a_bits = params
    modulus = 0
    while modulus.bit_length() != size_bits:
        p1 = generate_weak_prime(M, k_bits, a_bits)
        p2 = generate_weak_prime(M, k_bits, a_bits)
        modulus = p1 * p2
    return modulus

# ----- ROCA detection function (uses Pohlig-Hellman via SageZmod.log)
    -----

def detect_roca_vulnerability(n):
    params = derive_parameters(n.bit_length())
    if params is None:
        return False
    M = params[0]
    try:
        exponent_val = Zmod(M)(n).log(65537)
        print(f"Vulnerable key detected! Exponent a = {exponent_val}")
        return True
    except ValueError:
        print("Key is not vulnerable to ROCA.")
        return False

# ----- Demo / tests -----
if __name__ == "__main__":
    # Test a known weak key (generated with the same procedure)
    print()
    vulnerable_modulus = generate_weak_modulus(512)
    print("Testing vulnerable (generated) modulus:")
    detect_roca_vulnerability(vulnerable_modulus)
    print()
```

```
    # Test a secure key (random primes)
    secure_modulus = int(getPrime(300) * getPrime(300))
    print("Testing a random secure modulus:")
    detect_roca_vulnerability(secure_modulus)
    print()
```

```
(sage) root@DESKTOP-T0U67CD:/home/anshjaiin/roca# sage roca_detect.py

Testing vulnerable (generated) modulus:
Vulnerable key detected! Exponent a = 1016137529059735899

Testing a random secure modulus:
Key is not vulnerable to ROCA.
```

5 The Attack Principle

- After understanding how the primes are generated and how to detect vulnerable keys, the next step is factoring the modulus N.
- The ROCA attack employs a variant of Coppersmith's algorithm, refined by Howgrave–Graham (hereafter referred to as CHG).
- Coppersmith (univariate modular) finds all integer roots x_0 of a monic polynomial (a polynomial with leading coefficient 1) $f(x)$ modulo N provided the root is smaller than about $N^{1/d}$, where $d = \deg(f)$.

  In symbols: if
  $$f(x_0) \equiv 0 \pmod{N} \quad \text{and} \quad |x_0| \leq N^{1/d},$$
  then Coppersmith's method (via an LLL-constructed lattice) recovers x_0 in polynomial time.
- Howgrave–Graham's work gives the lattice bounds that make that statement rigorous and improves the concrete constants used in practice.

5.1 Mapping This Condition to the ROCA Attack Scenario

- We have $p = k \cdot M + r$ where the residue
  $$r = (65537^{\,a} \bmod M).$$
  Now, we will see that if we know a, we can use CHG to find the value of k. Then p will be known, and hence N can be factorized.
- Since $p \approx N^{1/2}$, roughly
  $$k \approx \frac{p}{M} \approx \frac{N^{1/2}}{M}.$$
  To apply the univariate Coppersmith bound, we require k to be smaller than the bound given by Howgrave–Graham for the constructed polynomial. After performing polynomial lifting and lattice construction, the bound on the small root k is effectively around $N^{1/4}$ in the ROCA scenario.
- Hence, we require
  $$k \lesssim N^{1/4}$$
  which implies
  $$\frac{N^{1/2}}{M} \lesssim N^{1/4} \quad \Longrightarrow \quad M \gtrsim N^{1/2}/N^{1/4} = N^{1/4}.$$

- In logarithmic form:

$$\log_2 M \gtrsim \frac{1}{4} \log_2 N.$$

- Polynomial lifting means constructing multiple higher–degree multiples of the base polynomial so that their integer roots modulo N remain unchanged, but the lifted polynomials contain more algebraic information. This enables a lattice reduction method (such as LLL) to recover the small root k.

5.2 Checking whether can we apply CHG

- Now, Checking whether the condition

$$\log_2 M \geq \frac{1}{4} \log_2 N$$

is satisfied or not

```python
from sympy import primerange, factorint, nextprime as next_prime
from decimal import Decimal, getcontext
from math import log2, ceil
from Crypto.Util.number import getRandomNBitInteger, isPrime as
    is_prime, getPrime

getcontext().prec = 200


# ----------------------------------------------------------
# Compute primorial P_n#
# ----------------------------------------------------------
def compute_primorial(n):
    primes = list(primerange(1, 10000))[:n]
    M = 1
    for p in primes:
        M *= p
    return M


# ----------------------------------------------------------
# Compute multiplicative order of g modulo M (pure Python)
# ----------------------------------------------------------
def multiplicative_order(g, M):
    # phi(M) =    (p_i - 1)
    primes = list(primerange(1, 10000))
    n = 0
    product = 1
    for p in primes:
        if product * p > M:
            break
        product *= p
        n += 1

    primes_used = primes[:n]
    phi = 1
    for p in primes_used:
        phi *= (p - 1)

    # factor phi(M)
    factors = factorint(phi)
```

```
41
42      # compute order by dividing out factors
43      t = phi
44      for q, e in factors.items():
45          for _ in range(e):
46              if pow(g, t // q, M) == 1:
47                  t //= q
48              else:
49                  break
50      return t
51
52
53  # ----------------------------------------------------------
54  # Derive ROCA parameters (pure Python)
55  # ----------------------------------------------------------
56  def derive_parameters(key_bits):
57      if 512 <= key_bits <= 960:
58          n = 39
59      elif 992 <= key_bits <= 1952:
60          n = 71
61      elif 1984 <= key_bits <= 3936:
62          n = 126
63      elif 3968 <= key_bits <= 4096:
64          n = 225
65      else:
66          return None
67
68      M = compute_primorial(n)
69
70      k_bits = key_bits // 2 - round(log2(M))
71
72      order = multiplicative_order(65537, M)
73      a_bits = ceil(log2(order))
74
75      return M, k_bits, a_bits
76
77
78  # ----------------------------------------------------------
79  # Generate prime with ROCA structure
80  # ----------------------------------------------------------
81  def generate_weak_prime(M, k_bits, a_bits):
82      while True:
83          k = getRandomNBitInteger(k_bits)
84          a = getRandomNBitInteger(a_bits)
85          p = k * M + pow(65537, a, M)
86          if is_prime(p):
87              return p
88
89
90  # ----------------------------------------------------------
91  # Generate N = p1 * p2 where p1, p2 are ROCA primes
92  # ----------------------------------------------------------
93  def generate_weak_modulus(size_bits):
94      params = derive_parameters(size_bits)
95      if params is None:
96          raise ValueError("Unsupported␣key␣size.")
97
98      M, k_bits, a_bits = params
```

```
99
100     while True:
101         p1 = generate_weak_prime(M, k_bits, a_bits)
102         p2 = generate_weak_prime(M, k_bits, a_bits)
103         N = p1 * p2
104         if N.bit_length() == size_bits:
105             return N, M
106
107
108 # ------------------------------------------------------------
109 # MAIN
110 # ------------------------------------------------------------
111 N, M = generate_weak_modulus(512)
112
113 if log2(M) >=  (1/4 * (log2(N))) :
114    print("YES")
```

```
PS C:\Users\desha> python -u "c:\Users\desha\Downloads\slide 22 - file4.py"
YES
```

- Hence, the condition is satisfied and we can use CHG to find k. If a was correctly guessed, the algorithm will find a solution, which allows us to recover p.
- In practice, there are roughly $2^{61.09}$ possibilities for a, which is too large to brute

5.3 Transforming the Problem to CHG

- We have
$$p = k \cdot M + (65537^{\,a} \bmod M).$$
Now, we define
$$f(x) = k \cdot M + (65537^{\,a} \bmod M) \pmod N$$
so the root $x_0 = k$ satisfies $f(x_0) \equiv 0 \pmod p$.
- There is a problem here
- We cannot use $\bmod p$ because p is unknown. It is the unknown divisor of N.
- Even though the true relation holds modulo p, since $p \mid N$:
$$f(x_0) \equiv 0 \pmod N \implies f(x_0) \equiv 0 \pmod p.$$
- Thus, we can safely construct the lattice modulo N, which is public.
- Now, the next step is to make the polynomial monic. CHG requires the polynomial to be monic — that is, the leading coefficient should be 1.
- We make it monic by multiplying with $M^{-1} \bmod N$:
$$f(x) = (x \cdot M + (65537^{\,a} \bmod M)) \cdot (M^{-1} \bmod N)$$
$$= x + (65537^{\,a} \bmod M) \cdot (M^{-1} \bmod N) \pmod N.$$
- This final monic modular polynomial is used for the CHG lattice construction.

5.4 CHG Parameters

- The CHG algorithm uses 6 parameters:
  1. The polynomial $f(x)$
  2. The modulus N
  3. A lower bound β
  4. A parameter m
  5. A parameter t
  6. An upper bound X

5.5 The Polynomial $f(x)$ and the Modulus N

- **The polynomial $f(x)$:**
  This is the modular polynomial we constructed earlier:

$$f(x) = k \cdot M + (65537^{\,a} \bmod M) \pmod{N}.$$

- **The modulus N:**
  N is the RSA modulus we aim to factorize:

$$N = p_1 \times p_2.$$

5.6 Lower Bound β

- β is a lower bound on the factor p of N:

$$p \geq N^{\beta}, \qquad 0 < \beta \leq 1.$$

- In practice, since RSA primes are roughly the same size, we most commonly set

$$\beta = 0.5.$$

  This corresponds to

$$p \approx q \approx N^{1/2}.$$

- But with $\beta = 0.5$, the algorithm will only recover the larger of the two primes, since both cannot exceed $N^{1/2}$.

5.7 Parameters m and t

- CHG constructs a lattice of size:

$$n = m + t$$

  where m : controls how many shifted multiples of $f(x)$ are used, t : controls how many powers of x are added to the lattice.

- Generally,

$$t = m + 1.$$

- These parameters affect the matrix dimension used by the LLL algorithm: If the matrix is too small $\rightarrow$ LLL may fail to find the root. If it is too large $\rightarrow$ computation becomes infeasible.

- For 512-bit key size, experimentation showed the optimal choice:

$$m = 5, \qquad t = m + 1 = 6.$$

5.8 The Upper Bound X

- The value X is the upper bound on the small root x_0 that the algorithm is searching for, i.e., the upper bound on k in our case.
- Recall that k has entropy of 37 bits for 512-bit key size. So, the upper bound is

$$X = 2^{37}$$

for the 512-bit key size.

- Now,Computing the value of

$$\frac{2 \times N^{1/2}}{M}$$

```python
from sympy import primerange, factorint, nextprime as next_prime
from decimal import Decimal, getcontext
from math import log2, ceil
from Crypto.Util.number import getRandomNBitInteger, isPrime as
    is_prime, getPrime

getcontext().prec = 200


# ----------------------------------------------------------
# Compute primorial P_n#
# ----------------------------------------------------------
def compute_primorial(n):
    primes = list(primerange(1, 10000))[:n]
    M = 1
    for p in primes:
        M *= p
    return M


# ----------------------------------------------------------
# Compute multiplicative order of g modulo M (pure Python)
# ----------------------------------------------------------
def multiplicative_order(g, M):
    # phi(M) =    (p_i - 1)
    primes = list(primerange(1, 10000))
    n = 0
    product = 1
    for p in primes:
        if product * p > M:
            break
        product *= p
        n += 1

    primes_used = primes[:n]
    phi = 1
    for p in primes_used:
        phi *= (p - 1)

    # factor phi(M)
    factors = factorint(phi)

    # compute order by dividing out factors
    t = phi
```

```
    for q, e in factors.items():
        for _ in range(e):
            if pow(g, t // q, M) == 1:
                t //= q
            else:
                break
    return t


# ----------------------------------------------------------
# Derive ROCA parameters (pure Python)
# ----------------------------------------------------------
def derive_parameters(key_bits):
    if 512 <= key_bits <= 960:
        n = 39
    elif 992 <= key_bits <= 1952:
        n = 71
    elif 1984 <= key_bits <= 3936:
        n = 126
    elif 3968 <= key_bits <= 4096:
        n = 225
    else:
        return None

    M = compute_primorial(n)

    k_bits = key_bits // 2 - round(log2(M))

    order = multiplicative_order(65537, M)
    a_bits = ceil(log2(order))

    return M, k_bits, a_bits


# ----------------------------------------------------------
# Generate prime with ROCA structure
# ----------------------------------------------------------
def generate_weak_prime(M, k_bits, a_bits):
    while True:
        k = getRandomNBitInteger(k_bits)
        a = getRandomNBitInteger(a_bits)
        p = k * M + pow(65537, a, M)
        if is_prime(p):
            return p


# ----------------------------------------------------------
# Generate N = p1 * p2 where p1, p2 are ROCA primes
# ----------------------------------------------------------
def generate_weak_modulus(size_bits):
    params = derive_parameters(size_bits)
    if params is None:
        raise ValueError("Unsupported key size.")

    M, k_bits, a_bits = params

    while True:
        p1 = generate_weak_prime(M, k_bits, a_bits)
```

```
        p2 = generate_weak_prime(M, k_bits, a_bits)
        N = p1 * p2
        if N.bit_length() == size_bits:
            return N, M


# ------------------------------------------------------------
# MAIN
# ------------------------------------------------------------
N, M = generate_weak_modulus(512)

value = (Decimal(2) * Decimal(N).sqrt()) / Decimal(M)

print("Value ", value)
print("log2(Value) ", float((value.ln() / Decimal(2).ln())))
```

```
(base) tanmaisai@hpenvy: $ python3 file5.py
Value = 172391664723.79213279827146452228991832241428675911504807139962701158632983496535573396646093803038488689986002034731132433853518905221436635037738902074224796448246378509988432713461836103991370444751
log2(Value) = 37.326899064404
```

- It is equal to
$$2^{37.5}.$$
- Hence,
$$X \approx \frac{2 \times N^{1/2}}{M}.$$

5.9 Lattice Construction in CHG

- **Goal:** To find small roots x_0 of a polynomial $f(x)$ modulo N, i.e.,
$$f(x_0) \equiv 0 \pmod{N}, \qquad |x_0| < X.$$
- The lattice is built using carefully chosen polynomials derived from $f(x)$.

(a) Shifted multiples of $f(x)$

$$f_i(x) = x^j \, N^i \, f(x)^m$$

where - i runs over the number of shifts, i.e., $i = 0, 1, 2, \ldots, m-1$ - j runs over powers of x in each shift, i.e., $j = 0, 1, 2, \ldots, \delta - 1$ - δ is the degree of the original polynomial $f(x)$ (often $\delta = 1$ for linear polynomials)

Purpose: Encoding of the modular structure of $f(x)$. Any small root x_0 will make these polynomials vanish modulo N.

(b) Powers of x

$$f_{i+m}(x) = x^i \, f(x)^m$$

where - i runs over the number of added powers for smallness, i.e., $i = 0, 1, 2, \ldots, t-1$.

Purpose: Enforce that the root is small, i.e., within the bound X. These rows ensure that after lattice reduction, the solution corresponds to an *actual integer root*, not just a modulo solution.

- Each polynomial becomes a row in the lattice. Scaling powers of x by X ensures the vector corresponding to the small root is short enough to be found by LLL. The lattice is square with dimension

$$n = \delta \cdot m + t$$

(the number of polynomials).

- Then, we apply LLL lattice reduction to the matrix. LLL finds short vectors in the lattice. One of these vectors corresponds to a polynomial that vanishes at the small root x_0 over the integers, not just modulo N.

6 Optimizations

6.1 Reducing the Search Space for a

- We have shown earlier that the modulus N has the same form as its prime factors:

$$N = k_3 M + (65537^{\,a_3} \bmod M)$$

and the exponents satisfy

$$a_3 \equiv a_1 + a_2 \pmod{|G|}$$

where G is the subgroup of $\mathbb{Z}_M^*$ generated by 65537.

- Without optimization, we would need to search for a_1 or a_2 in the full range

$$[0,\, a_3].$$

- However, we can **halve the search space** for possible values of a_3:

$$a_1,\, a_2 \in \left[\frac{a_3}{2}, \frac{a_3 + |G|}{2}\right].$$

- Since

$$a_3 \equiv a_1 + a_2 \pmod{|G|},$$

the minimum combination occurs when

$$a_1 = a_2 = \frac{a_3}{2},$$

and the maximum combination occurs when

$$a_1 = a_2 = \frac{a_3 + |G|}{2}.$$

- For all other cases, only one of a_1, a_2 will lie in the interval, and the attack will recover whichever prime corresponds to the exponent we guessed correctly.

- Hence, instead of searching the whole interval $[0,\, a_3]$, we only search half the space. This reduces the brute-force exponent search by **50%**.

6.2 β Parameter Issue After Reduction of the Search Space for a

- Recall that the parameter β was set to 0.5, assuming both primes are roughly half the size of N. This implies that the algorithm will only recover the *larger* of the two primes, since both cannot exceed $N^{1/2}$.
- However, this causes an issue when only the *smaller* prime lies inside the search interval.
- For instance: If $p_1 > p_2$, and
$$a_1 \notin \left[\frac{a_3}{2}, \frac{a_3 + |G|}{2}\right],$$
then the algorithm only searches in the interval
$$\left[\frac{a_3}{2}, \frac{a_3 + |G|}{2}\right].$$
But
$$a_2 \in \left[\frac{a_3}{2}, \frac{a_3 + |G|}{2}\right],$$
so only p_2 could be recovered, because its corresponding exponent lies in the search interval.
- However, due to the restriction imposed by the lower bound $\beta = 0.5$, **only the larger prime can be recovered**. Since p_2 is the smaller prime, the algorithm will not recover *either* p_1 or p_2.
- The fix is simple: use a smaller value for β (for example, $\beta = 0.1$).

6.3 Optimizing the Value M

- The next improvement focuses on optimizing M, the primorial used to generate weak primes.
- Recall the CHG (Coppersmith–Howgrave–Graham) method works only if:
$$\log_2 M \;\geq\; \frac{1}{4}\log_2 N.$$
- We have seen earlier that for 512-bit key size,
$$\log_2 M \approx 219 \qquad \text{and} \qquad \frac{1}{4}\log_2 N \approx 128,$$
i.e., the chosen M is larger than necessary, making the search for finding a_3 slower.
- Now we have to find the optimal M' so that it satisfies the following constraints:
  1. Structure of primes should be preserved.
  2. CHG bound must hold.
  3. Running time must be minimal.

1. Structure of primes should be preserved

- To satisfy this constraint, M' should be a divisor of M.
- If $M' \mid M$, then:
$$65537^{a_3} \bmod M' = 65537^{a_3} \bmod M \mod M'.$$

2. CHG bound must hold

- To apply CHG, we must satisfy:

$$\log_2 M' \geq \frac{1}{4} \log_2 N.$$

- Since for a 512-bit key size,

$$\frac{1}{4} \log_2 N = \frac{512}{4} = 128,$$

only values of

$$M' \geq 2^{128}$$

are usable.

3. Running time must be minimal

- Increasing M' reduces the number of possible exponents a_3, thereby decreasing the total number of guesses required because fewer group elements exist.
- However, making M' too large slows down each individual attempt. In the CHG lattice construction, the coefficients of the polynomial are roughly proportional to M'. Larger coefficients make the lattice vectors bigger in magnitude. Bigger vectors require more computational effort to reduce using LLL.
- Thus, the choice of M' represents a **trade-off** between:
  - minimizing the number of required guesses, and
  - minimizing the time per CHG attempt.

6.4 Optimizing the Values M' and m

- The choice of the parameter m depends indirectly on the value of M'. A larger M' means more information about the prime p is known, and hence a smaller lattice is sufficient to recover the unknown k, and thus a smaller m can be used.
- Reducing m results in a smaller lattice, which makes the LLL lattice reduction faster. Consequently, it decreases the time per attempt.
- However, if m is too small, the lattice may not contain sufficient information to recover the small root k. Even if the correct value of a_3 is provided, CHG may fail to find a solution, which reduces the overall success rate of the attack.
- Thus, the choice of m represents a **trade-off** between:
  - minimizing the time per attempt, and
  - maximizing the success rate.
- This optimization work has to be done for each key size, but it only needs to be done once.

Optimal M' and m Choices by Key Size

Key Size	**Chosen M'**	**Chosen m**
512 – 960	`0x1b3e6c9433a7735fa5fc479ffe4027e13bea`	5
992 – 1952	`0x24683144f41188c2b1d6a217f81f112888e4e6513c43f 3f60e72af8bd9728807483425d1e`	4
1984 – 3936	`0x16928dc3e47b44daf289a60e80e1fc6bd7648d7ef60d 1890f3e0a9455efe0abdb7a748131413cebd2e36a76a3 55c1b664be462e115ac330f9c13344f8f3d1034a02c23396e6`	7

7 Implementation of the Return of Coppersmith's Attack or ROCA using CHG Algorithm

```
    from sage.all_cmdline import *
from sage.all import *


def hg_univariate_redux(pol, modulus, beta, mm, tt, XX):

    dd = pol.degree()
    nn = dd * mm + tt

    if not 0 < beta <= 1:
        raise ValueError("beta should belongs in [0, 1]")

    if not pol.is_monic():
        raise ArithmeticError("Polynomial must be monic.")

    polZ = pol.change_ring(ZZ)
    x = polZ.parent().gen()

    gg = []
    for ii in range(mm):
        for jj in range(dd):
            gg.append((x * XX) ** jj * modulus ** (mm - ii) * polZ(x *
                XX) ** ii)
    for ii in range(tt):
        gg.append((x * XX) ** ii * polZ(x * XX) ** mm)

    BB = Matrix(ZZ, nn)
    for ii in range(nn):
        for jj in range(ii + 1):
            BB[ii, jj] = gg[ii][jj]

    BB = BB.LLL()

    new_pol = 0
    for ii in range(nn):
        new_pol += x ** ii * BB[0, ii] / XX ** ii #reverse the scaling
            and reconstruct the polynomial

    possible = new_pol.roots()
    roots = []

    for root in possible:
        if root[0].is_integer():
            result = polZ(ZZ(root[0]))
            if gcd(modulus, result) >= modulus ** beta:    #f(x0)    p
                (mod N) so gcd (N, f(x0))= gcd (pq, p)=p
                roots.append(ZZ(root[0]))

    return roots



def roca_solve_instance(M, n, a, m):

```

```
    from __main__ import hg_univariate_redux

    base = int(65537)
    known = int(pow(base, a, M) * inverse_mod(M, n))

    F = PolynomialRing(Zmod(n), implementation='NTL', names=('x',))
    (x,) = F._first_ngens(1)
    pol = x + known

    beta = 0.1
    t = m + 1
    XX = floor(2 * n**0.5 / M)

    roots = hg_univariate_redux(pol, n, beta, m, t, XX)

    for k in roots:
        p = int(k * M + pow(base, a, M))
        if n % p == 0:
            return p, n // p


def roca_factor_main(n):

    keySize = n.bit_length()

    if keySize <= 960:
        M_prime = 0x1b3e6c9433a7735fa5fc479ffe4027e13bea
        m = 5

    elif 992 <= keySize <= 1952:
        M_prime = 0
            x24683144f41188c2b1d6a217f81f12888e4e6513c43f3f60e72af8bd9728807483425d1

        m = 4

    elif 1984 <= keySize <= 3936:
        M_prime = 0
            x16928dc3e47b44daf289a60e80e1fc6bd7648d7ef60d1890f3e0a9455efe0abdb7a7481

        m = 7

    elif 3968 <= keySize <= 4096:
        return None

    else:
        print("Invalid key size: {}".format(keySize))
        return None

    a3 = Zmod(M_prime)(n).log(65537) # n == 65537^a3 (mod m_prime)
    print(f"Exponent a3 such that 65537^a3    n (mod M_prime) is: {a3}
        ")

    order = Zmod(M_prime)(65537).multiplicative_order()
    inf = a3 // 2
    sup = (a3 + order) // 2

    order = Zmod(M_prime)(65537).multiplicative_order()
    inf = a3 // 2
```

```
    sup = (a3 + order) // 2

    print(f"Order of the group generated by 65537 modulo M_prime (|G'|): {order}")

    print(f"Size in bits of the group (entropy of a): {log(order, 2).n()}")

    print(f"Search range for exponent a1 or a2: [{inf}, {sup}]")
    print(f"Range size: {sup - inf}")


    chunk_size = 10000

    for inf_a in range(inf, sup, chunk_size):

        inputs = [((M_prime, n, a, m), {}) for a in range(inf_a, inf_a + chunk_size)]

        from sage.parallel.multiprocessing_sage import parallel_iter
        from multiprocessing import cpu_count

        for k, val in parallel_iter(cpu_count(), roca_solve_instance, inputs):
            if val:
                p, q = val
                print("We've factorized the modulus N:\n p = {}\n q = {}".format(p, q))
                return val

def compute_primorial(count):
    product = 1
    prime_cursor = 1
    for _ in range(count):
        prime_cursor = next_prime(prime_cursor)
        product *= prime_cursor
    return int(product)

def derive_parameters(key_bits):
    if key_bits <= 960:
        n = 39
    elif 992 <= key_bits <= 1952:
        n = 71
    elif 1984 <= key_bits <= 3936:
        n = 126
    elif 3968 <= key_bits <= 4096:
        n = 225
    else:
        return None
    M = compute_primorial(n)
    k_bits = key_bits // 2 - round(log(M, 2))
    g_order = Zmod(M)(65537).multiplicative_order()
    a_bits = ceil(log(g_order, 2))
    return M, k_bits, a_bits

def detect_roca_vulnerability(n):
    params = derive_parameters(n.bit_length())
    if params is None:
```

```
        return False
    M = params[0]
    try:
        exponent_val = Zmod(M)(n).log(65537)
        print(f"Vulnerable key detected!  Exponent a3 satisfying 65537^
            a3    n (mod M): {exponent_val}")
        return True
    except ValueError:
        print("Key is not vulnerable to ROCA.")
        return False


if __name__ == "__main__":
    p =
        80688738291820833650844741016523373313635060001251156496219948915457811770063

    q =
        69288134094572876629045028069371975574660226148748274586674507084213286357069


    n = p * q

    detect_roca_vulnerability(n)


    roca_factor_main(n)
```

```
(sage) root@DESKTOP-T0U67CD:/home/anshjaiin/roca# sage roca.py
Vulnerable key detected!  Exponent a3 satisfying 65537^a3 ≡ n (mod M): 701415872538123025
Exponent a3 such that 65537^a3 ≡ n (mod M_prime) is: 342625
Order of the group generated by 65537 modulo M_prime (|G'|): 1201200
Size in bits of the group (entropy of a): 20.1960449493319
Search range for exponent a1 or a2: [171312, 771912]
Range size: 600600
We've factorized the modulus N:
 p = 69288134094572876629045028069371975574660226148748274586674507084213286357069
 q = 80688738291820833650844741016523373313635060001251156496219948915457811770063
```

8 Countermeasures and Mitigation

The ROCA vulnerability stems from a fundamental flaw in Infineon's deterministic prime-generation process. Effective mitigation therefore requires eliminating the structure that enabled the CHG-based factorization attack.

8.1 Eliminating the Vulnerable Prime Structure

ROCA-prone primes satisfy:

$$p = kM + (65537^a \bmod M),$$

which restricts primes to a predictable subset of all primes of their size.

Mitigation requires completely removing this structured construction. Vendors must:

- discontinue Infineon's RSALib prime generator,
- adopt standard randomized prime generation,
- avoid residue-based patterns or other deterministic constraints.

8.2 Firmware and Library Updates

Infineon released updated firmware that replaces the flawed prime-generation routine. The updated versions:

- generate primes uniformly at random,
- remove primorial-based residues,
- eliminate the $65537^a \bmod M$ structure.

Hardware devices using older firmware must be updated or replaced.

8.3 Key Revocation and Regeneration

All RSA keys produced by vulnerable versions of RSALib must be considered compromised. Mitigation actions include:

- revoking affected certificates,
- regenerating RSA key pairs with updated software,
- reissuing authentication credentials or TPM bindings.

Large-scale key rotation was required in many national and enterprise systems.

8.4 Vendor and Ecosystem Response

ROCA affected:

- TPMs,
- smartcards and government e–ID cards,
- authentication tokens,
- secure identity and signing systems.

Vendors responded by issuing advisories, distributing patches, and in some cases replacing affected hardware.

8.5 Long-Term Lessons for Cryptographic Engineering

ROCA illustrates several key principles:

1. **Randomness is essential.** Structured primes undermine RSA security.
2. **Security by obscurity fails.** Proprietary designs must still be audited.
3. **Large key sizes cannot compensate for structural weaknesses.**
4. **Hardware is not automatically secure.** TPMs and smartcards can be flawed.
5. **Independent review of cryptographic libraries is mandatory.**

8.6 Summary

Mitigation requires eliminating structured prime generation, updating firmware, revoking vulnerable keys, and ensuring transparent and thoroughly audited cryptographic implementations. ROCA demonstrates that RSA's security depends critically on secure prime generation, not solely on mathematical principles.

9 Conclusion

The ROCA vulnerability illustrates how a subtle flaw in RSA prime generation can compromise the security of millions of cryptographic systems, despite the mathematical robustness of RSA itself. Infineon's deterministic prime-generation method,

$$p = kM + (65537^a \bmod M),$$

restricted primes to a predictable subset and leaked structural information into the RSA modulus.

Once this structure was reverse-engineered, researchers constructed a practical Coppersmith–Howgrave–Graham (CHG) attack capable of recovering a prime factor of N by treating part of the prime as a small unknown. With optimizations such as reduced exponent search ranges, optimized primorials, and tuned lattice parameters, the attack became feasible on ordinary hardware for many key sizes.

ROCA had significant real-world impact, affecting TPMs, smartcards, identity tokens, and government-issued credentials. The vulnerability underscores several important lessons for cryptographic engineering:

- secure randomness is essential, especially in prime generation;
- structural constraints in primes can be catastrophic;
- proprietary cryptographic implementations must be independently audited;
- large RSA key sizes cannot compensate for weak key structure.

Ultimately, ROCA demonstrates that cryptographic security depends critically on implementation quality. Even mathematically sound primitives such as RSA fail when the underlying key-generation algorithms introduce unintended structure. Ensuring transparency, rigorous testing, and robust randomness is essential to preventing similar vulnerabilities in the future.

References

The Return of Coppersmith's Attack: Practical Factorization of Widely Used RSA Moduli. Available at: `https://crocs.fi.muni.cz/_media/public/papers/nemec_roca_ccs17_preprint.pdf`

`https://github.com/mimoo/RSA-and-LLL-attacks`

`https://en.wikipedia.org/wiki/Pohlig%E2%80%93Hellman_algorithm`

`https://bitsdeep.com/posts/analysis-of-the-roca-vulnerability/`